

Module 4: Pandas Series

Learning Objectives

- Introduction to Pandas library
- Understanding Series data structure
- Series creation and manipulation
- Indexing and selection operations
- Basic operations on Series

Topics Covered

1. Introduction to Pandas and its ecosystem
2. Creating Series objects
3. Series attributes and methods
4. Indexing and selection
5. Series operations and arithmetic
6. Handling missing values in Series
7. Series methods for data analysis
8. String operations on Series
9. Date and time operations
10. Series aggregation and statistics
11. Series plotting basics
12. File I/O Operations

Exercises

- Series creation and manipulation exercises
- Data analysis problems using Series

1. Introduction to Pandas and its Ecosystem

```
In [ ]: import pandas as pd
import numpy as np

# Check pandas version
print(f"Pandas version: {pd.__version__}")

# Pandas ecosystem overview
print("Pandas is built on top of NumPy and provides:")
print("- High-performance data structures")
print("- Data analysis tools")
print("- Data manipulation capabilities")
print("- Time series functionality")
print("- Easy data import/export")
```

```

# Key pandas data structures
print(f"\nMain pandas data structures:")
print(f"1. Series – 1D labeled array")
print(f"2. DataFrame – 2D labeled data structure")
print(f"3. Index – Immutable array implementing an ordered, sliceable set")

# Import common libraries in pandas ecosystem
print(f"\nCommon libraries in pandas ecosystem:")
print(f"- NumPy: Numerical computing foundation")
print(f"- Matplotlib: Plotting and visualization")
print(f"- Seaborn: Statistical data visualization")
print(f"- Scikit-learn: Machine learning")
print(f"- Jupyter: Interactive computing")

```

2. Creating Series Objects

```

In [ ]: # Creating Series from lists
data = [10, 20, 30, 40, 50]
series1 = pd.Series(data)
print(f"Series from list:\n{series1}")

# Creating Series with custom index
index_labels = ['A', 'B', 'C', 'D', 'E']
series2 = pd.Series(data, index=index_labels)
print(f"\nSeries with custom index:\n{series2}")

# Creating Series from dictionary
data_dict = {'Apple': 3.5, 'Banana': 2.1, 'Orange': 4.0, 'Grape': 5.2}
series3 = pd.Series(data_dict)
print(f"\nSeries from dictionary:\n{series3}")

# Creating Series from NumPy array
np_array = np.array([1, 2, 3, 4, 5])
series4 = pd.Series(np_array, index=['a', 'b', 'c', 'd', 'e'])
print(f"\nSeries from NumPy array:\n{series4}")

# Creating Series with scalar values
series5 = pd.Series(7, index=['x', 'y', 'z'])
print(f"\nSeries with scalar value:\n{series5}")

# Creating Series with different data types
mixed_data = [1, 2.5, 'hello', True, None]
series6 = pd.Series(mixed_data, index=['int', 'float', 'str', 'bool', 'no'])
print(f"\nSeries with mixed data types:\n{series6}")
print(f>Data type: {series6.dtype}")

# Series with date index
dates = pd.date_range('2024-01-01', periods=5, freq='D')
series7 = pd.Series([100, 110, 105, 120, 115], index=dates)
print(f"\nSeries with date index:\n{series7}")

```

3. Series Attributes and Methods

```

In [ ]: # Create a sample Series
data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
index_labels = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']

```

```

series = pd.Series(data, index=index_labels, name='Sample Data')

print(f"Series:\n{series}")

# Basic attributes
print(f"\n--- Series Attributes ---")
print(f"Name: {series.name}")
print(f"Index: {series.index}")
print(f"Values: {series.values}")
print(f>Data type: {series.dtype}")
print(f"Size: {series.size}")
print(f"Shape: {series.shape}")
print(f"Memory usage: {series.memory_usage()} bytes")

# Index attributes
print(f"\n--- Index Attributes ---")
print(f"Index values: {series.index.values}")
print(f"Index name: {series.index.name}")
print(f"Index is unique: {series.index.is_unique}")

# Data access methods
print(f"\n--- Data Access Methods ---")
print(f"First 3 values: {series.head(3)}")
print(f>Last 3 values: {series.tail(3)}")
print(f"Sample values: {series.sample(3)}")

# Statistical methods
print(f"\n--- Statistical Methods ---")
print(f"Count: {series.count()}")
print(f"Sum: {series.sum()}")
print(f"Mean: {series.mean():.2f}")
print(f"Median: {series.median():.2f}")
print(f"Mode: {series.mode().values}")
print(f"Standard deviation: {series.std():.2f}")
print(f"Variance: {series.var():.2f}")
print(f"Min: {series.min()}")
print(f"Max: {series.max()}")
print(f"Range: {series.max() - series.min()}")

# Data type methods
print(f"\n--- Data Type Methods ---")
print(f"Is numeric: {series.dtype in ['int64', 'float64']}")
print(f"Has null values: {series.isnull().any()}")
print(f"Number of null values: {series.isnull().sum()}")

# Sorting methods
print(f"\n--- Sorting Methods ---")
print(f"Sorted ascending:\n{series.sort_values()}")
print(f"Sorted descending:\n{series.sort_values(ascending=False)}")
print(f"Sorted by index:\n{series.sort_index()}")

# Unique values
print(f"\n--- Unique Values ---")
print(f"Unique values: {series.unique()}")
print(f"Number of unique values: {series.nunique()}")
print(f"Value counts:\n{series.value_counts()}")

```

4. Indexing and Selection

```
In [ ]: # Create a comprehensive Series for indexing examples
data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
index_labels = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
series = pd.Series(data, index=index_labels, name='Sample Data')

print(f"Original Series:\n{series}")

# Position-based indexing (iloc)
print(f"\n--- Position-based Indexing (iloc) ---")
print(f"First element: {series.iloc[0]}")
print(f>Last element: {series.iloc[-1]}")
print(f"First 3 elements: {series.iloc[:3]}")
print(f>Last 3 elements: {series.iloc[-3:]}")
print(f"Elements at positions 1, 3, 5: {series.iloc[[1, 3, 5]]}")

# Label-based indexing (loc)
print(f"\n--- Label-based Indexing (loc) ---")
print(f"Element with label 'C': {series.loc['C']}")
print(f"Elements with labels 'A', 'C', 'E': {series.loc[['A', 'C', 'E']]}")
print(f"Elements from 'B' to 'F': {series.loc['B':'F']}")
print(f"Elements from 'D' to end: {series.loc['D':]}")

# Boolean indexing
print(f"\n--- Boolean Indexing ---")
print(f"Elements > 50: {series[series > 50]}")
print(f"Elements between 30 and 70: {series[(series >= 30) & (series <= 70)]}")
print(f"Even elements: {series[series % 2 == 0]}")

# Using .at and .iat for single value access
print(f"\n--- Single Value Access ---")
print(f"Value at position 2: {series.iat[2]}")
print(f"Value at label 'F': {series.at['F']}")

# Setting values
series_copy = series.copy()
series_copy.loc['A'] = 15
series_copy.iloc[1] = 25
print(f"\nAfter modification:\n{series_copy}")
```

5. Series Operations and Arithmetic

```
In [ ]: # Create two Series for operations
s1 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
s2 = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
s3 = pd.Series([100, 200, 300], index=['a', 'b', 'c']) # Different length

print(f"Series 1:\n{s1}")
print(f"Series 2:\n{s2}")
print(f"Series 3 (different length):\n{s3}")

# Basic arithmetic operations
print(f"\n--- Arithmetic Operations ---")
print(f>Addition: {s1 + s2}")
print(f>Subtraction: {s2 - s1}")
print(f>Multiplication: {s1 * s2}")
print(f>Division: {s2 / s1}")
print(f>Power: {s1 ** 2}")
```

```

# Operations with different indices (alignment)
print(f"\n--- Operations with Different Indices ---")
print(f"s1 + s3 (aligned):\n{s1 + s3}")
print(f"s1 * 10 (scalar):\n{s1 * 10}")

# Comparison operations
print(f"\n--- Comparison Operations ---")
print(f"s1 > 2:\n{s1 > 2}")
print(f"s1 == s2:\n{s1 == s2}")
print(f"s1 >= 3:\n{s1 >= 3}")

# Logical operations
print(f"\n--- Logical Operations ---")
mask1 = s1 > 2
mask2 = s1 < 5
print(f"mask1 (s1 > 2): {mask1}")
print(f"mask2 (s1 < 5): {mask2}")
print(f"mask1 AND mask2: {mask1 & mask2}")
print(f"mask1 OR mask2: {mask1 | mask2}")

# String operations (if Series contains strings)
text_series = pd.Series(['hello', 'world', 'python', 'pandas'], index=['a', 'b', 'c', 'd'])
print(f"\n--- String Operations ---")
print(f"Original: {text_series}")
print(f"Uppercase: {text_series.str.upper()}")
print(f"Length: {text_series.str.len()}")
print(f"Contains 'o': {text_series.str.contains('o')}")
print(f"Starts with 'p': {text_series.str.startswith('p')}")

# Mathematical functions
print(f"\n--- Mathematical Functions ---")
print(f"Square root: {np.sqrt(s1)}")
print(f"Exponential: {np.exp(s1)}")
print(f"Logarithm: {np.log(s1)}")
print(f"Absolute value: {np.abs(s1 - 3)}")

# Aggregation functions
print(f"\n--- Aggregation Functions ---")
print(f"Sum: {s1.sum()}")
print(f"Mean: {s1.mean():.2f}")
print(f"Standard deviation: {s1.std():.2f}")
print(f"Min: {s1.min()}")
print(f"Max: {s1.max()}")
print(f"Product: {s1.prod()}")

```

6. Handling Missing Values in Series

```

In [ ]: # Create Series with missing values
data_with_nan = [1, 2, np.nan, 4, 5, np.nan, 7, 8, 9, 10]
index_with_nan = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
series_with_nan = pd.Series(data_with_nan, index=index_with_nan)

print(f"Series with missing values:\n{series_with_nan}")

# Check for missing values
print(f"\n--- Missing Value Detection ---")
print(f"Has any missing values: {series_with_nan.isnull().any()}")
print(f"Number of missing values: {series_with_nan.isnull().sum()}")

```

```

print(f"Missing value positions: {series_with_nan.isnull()}")
print(f"Non-missing value positions: {series_with_nan.notnull()}")

# Drop missing values
print(f"\n--- Dropping Missing Values ---")
dropped = series_with_nan.dropna()
print(f"After dropping NaN values:\n{dropped}")

# Fill missing values
print(f"\n--- Filling Missing Values ---")
filled_zero = series_with_nan.fillna(0)
print(f"Filled with 0:\n{filled_zero}")

filled_mean = series_with_nan.fillna(series_with_nan.mean())
print(f"Filled with mean ({series_with_nan.mean():.2f}):\n{filled_mean}")

filled_forward = series_with_nan.fillna(method='ffill')
print(f"Forward fill:\n{filled_forward}")

filled_backward = series_with_nan.fillna(method='bfill')
print(f"Backward fill:\n{filled_backward}")

# Interpolation
print(f"\n--- Interpolation ---")
interpolated = series_with_nan.interpolate()
print(f"Linear interpolation:\n{interpolated}")

# Operations with missing values
print(f"\n--- Operations with Missing Values ---")
print(f"Sum (ignores NaN): {series_with_nan.sum()}")
print(f"Mean (ignores NaN): {series_with_nan.mean():.2f}")
print(f"Count (non-NaN values): {series_with_nan.count()}")

# Replace specific values
print(f"\n--- Replacing Values ---")
replaced = series_with_nan.replace([np.nan], 999)
print(f"Replaced NaN with 999:\n{replaced}")

# Working with None values
series_with_none = pd.Series([1, 2, None, 4, 5, None, 7])
print(f"\nSeries with None values:\n{series_with_none}")
print(f"None values are treated as NaN: {series_with_none.isnull().sum()}")

```

7. Series Methods for Data Analysis

```

In [ ]: # Create comprehensive Series for data analysis examples
np.random.seed(42)
data = np.random.normal(100, 15, 100) # Normal distribution with mean=100
index = pd.date_range('2024-01-01', periods=100, freq='D')
sales_data = pd.Series(data, index=index, name='Daily Sales')

# Add some missing values for demonstration
sales_data.iloc[10:15] = np.nan
sales_data.iloc[50:55] = np.nan

print(f"Sales Data Series (first 10 values):\n{sales_data.head(10)}")
print(f"\nSeries info:")
print(f"Length: {len(sales_data)}")

```

```

print(f"Data type: {sales_data.dtype}")
print(f"Memory usage: {sales_data.memory_usage()} bytes")

# Basic data analysis methods
print(f"\n--- Basic Data Analysis Methods ---")
print(f"Count (non-null values): {sales_data.count()}")
print(f"Sum: {sales_data.sum():.2f}")
print(f"Mean: {sales_data.mean():.2f}")
print(f"Median: {sales_data.median():.2f}")
print(f"Mode: {sales_data.mode().values}")
print(f"Standard deviation: {sales_data.std():.2f}")
print(f"Variance: {sales_data.var():.2f}")
print(f"Min: {sales_data.min():.2f}")
print(f"Max: {sales_data.max():.2f}")
print(f"Range: {sales_data.max() - sales_data.min():.2f}")

# Percentile analysis
print(f"\n--- Percentile Analysis ---")
print(f"25th percentile (Q1): {sales_data.quantile(0.25):.2f}")
print(f"50th percentile (Q2/Median): {sales_data.quantile(0.5):.2f}")
print(f"75th percentile (Q3): {sales_data.quantile(0.75):.2f}")
print(f"90th percentile: {sales_data.quantile(0.9):.2f}")
print(f"95th percentile: {sales_data.quantile(0.95):.2f}")
print(f"99th percentile: {sales_data.quantile(0.99):.2f}")

# Skewness and kurtosis
print(f"\n--- Distribution Shape ---")
print(f"Skewness: {sales_data.skew():.4f}")
print(f"Kurtosis: {sales_data.kurtosis():.4f}")

# Data quality analysis
print(f"\n--- Data Quality Analysis ---")
print(f"Missing values: {sales_data.isnull().sum()}")
print(f"Missing percentage: {(sales_data.isnull().sum() / len(sales_data))}")
print(f"Duplicate values: {sales_data.duplicated().sum()}")
print(f"Unique values: {sales_data.nunique()}")

# Rolling statistics
print(f"\n--- Rolling Statistics ---")
print(f"7-day rolling mean (first 10):\n{sales_data.rolling(window=7).mean().head(10)}")
print(f"30-day rolling mean (first 10):\n{sales_data.rolling(window=30).mean().head(10)}")
print(f"7-day rolling std (first 10):\n{sales_data.rolling(window=7).std().head(10)}")

# Expanding statistics
print(f"\n--- Expanding Statistics ---")
print(f"Expanding mean (first 10):\n{sales_data.expanding().mean().head(10)}")
print(f"Expanding sum (first 10):\n{sales_data.expanding().sum().head(10)}")

# Data transformation methods
print(f"\n--- Data Transformation Methods ---")
print(f"Log transformation: {np.log(sales_data.dropna()).head()}")
print(f"Square root transformation: {np.sqrt(sales_data.dropna()).head()}")
print(f"Standardized values (z-score):\n{(sales_data - sales_data.mean()) / sales_data.std().head()}")
print(f"Normalized values (0-1):\n{(sales_data - sales_data.min()) / (sales_data.max() - sales_data.min()).head()}")

# Data binning
print(f"\n--- Data Binning ---")
bins = [0, 80, 100, 120, 200]
labels = ['Low', 'Medium', 'High', 'Very High']
sales_binned = pd.cut(sales_data, bins=bins, labels=labels, include_lowes=True)

```



```

print(f"Binned sales data:\n{sales_binned.value_counts()}")

# Outlier detection
print(f"\n--- Outlier Detection ---")
Q1 = sales_data.quantile(0.25)
Q3 = sales_data.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = sales_data[(sales_data < lower_bound) | (sales_data > upper_bo
print(f"Outliers (IQR method): {len(outliers)} values")
print(f"Outlier values: {outliers.values}")

# Data sampling
print(f"\n--- Data Sampling ---")
print(f"Random sample (5 values):\n{sales_data.sample(5)}")
print(f"Systematic sample (every 10th value):\n{sales_data.iloc[:,10].hea

# Data ranking
print(f"\n--- Data Ranking ---")
print(f"Ranked values (first 10):\n{sales_data.rank().head(10)}")
print(f"Ranked values (descending, first 10):\n{sales_data.rank(ascending

```

8. String Operations on Series

```

In [ ]: # Create Series with string data for string operations
names = ['John Smith', 'jane doe', 'BOB WILSON', 'Alice Johnson', 'mike b
ages = [25, 30, 35, 28, 42, 33, 29, 31]
cities = ['New York', 'los angeles', 'CHICAGO', 'Houston', 'phoenix', 'Ph
emails = ['john@email.com', 'jane.doe@company.org', 'bob.wilson@test.net'

# Create Series objects
names_series = pd.Series(names, name='Names')
ages_series = pd.Series(ages, name='Ages')
cities_series = pd.Series(cities, name='Cities')
emails_series = pd.Series(emails, name='Emails')

print(f"Names Series:\n{names_series}")
print(f"\nCities Series:\n{cities_series}")
print(f"\nEmails Series:\n{emails_series}")

# Basic string operations
print(f"\n--- Basic String Operations ---")
print(f"Uppercase names:\n{names_series.str.upper()}")
print(f"Lowercase names:\n{names_series.str.lower()}")
print(f"Title case names:\n{names_series.str.title()}")
print(f"Capitalize names:\n{names_series.str.capitalize()}")

# String length and character operations
print(f"\n--- String Length and Character Operations ---")
print(f"Length of each name:\n{names_series.str.len()}")
print(f"First character of each name:\n{names_series.str[0]}")
print(f>Last character of each name:\n{names_series.str[-1]}")
print(f"Characters 2-5 of each name:\n{names_series.str[1:5]}")

# String splitting and joining
print(f"\n--- String Splitting and Joining ---")
print(f"Split names by space:\n{names_series.str.split(' ')}")

```



```

print(f"First name only:\n{name_series.str.split(' ').str[0]}")
print(f"Last name only:\n{name_series.str.split(' ').str[-1]}")
print(f"Join with underscore:\n{name_series.str.replace(' ', '_')}")

# String searching and pattern matching
print(f"\n--- String Searching and Pattern Matching ---")
print(f"Names containing 'John':\n{name_series.str.contains('John', case=)}")
print(f"Names starting with 'J':\n{name_series.str.startswith('J', case=)}")
print(f"Names ending with 'son':\n{name_series.str.endswith('son', case=)}")
print(f"Names matching pattern 'J.*':\n{name_series.str.match('J.*', c)}")

# String replacement
print(f"\n--- String Replacement ---")
print(f"Replace 'John' with 'Jonathan':\n{name_series.str.replace('John', 'Jonathan')}")
print(f"Replace multiple patterns:\n{name_series.str.replace(['John', 'Jon'], 'John')}")
print(f"Remove spaces:\n{name_series.str.replace(' ', '')}")

# Regular expressions
print(f"\n--- Regular Expressions ---")
print(f"Extract first word (first name):\n{name_series.str.extract(r'^(\w+)')}")
print(f"Extract last word (last name):\n{name_series.str.extract(r'(\w+)$')}")
print(f"Find all words:\n{name_series.str.findall(r'\w+')}")
print(f"Count vowels in each name:\n{name_series.str.count(r'[aeiouAEIOU]}")

# Email operations
print(f"\n--- Email Operations ---")
print(f"Email domains:\n{emails_series.str.extract(r'@(.+)')}")
print(f"Email usernames:\n{emails_series.str.extract(r'^(.+)@')}")
print(f"Emails from 'email.com':\n{emails_series.str.contains('@email.com')}")
print(f"Replace domain:\n{emails_series.str.replace(r'@.*', '@newdomain.com')}")

# String padding and alignment
print(f"\n--- String Padding and Alignment ---")
print(f"Pad names to 15 characters (right):\n{name_series.str.pad(15, side='right')}")
print(f"Pad names to 15 characters (left):\n{name_series.str.pad(15, side='left')}")
print(f"Center names in 15 characters:\n{name_series.str.center(15)}")
print(f"Pad with zeros to 10 characters:\n{name_series.str.zfill(10)}")

# String case operations
print(f"\n--- String Case Operations ---")
print(f"Swap case:\n{name_series.str.swapcase()}")
print(f"Is lowercase:\n{name_series.str.islower()}")
print(f"Is uppercase:\n{name_series.str.isupper()}")
print(f"Is title case:\n{name_series.str.istitle()}")

# String validation
print(f"\n--- String Validation ---")
print(f"Is alphabetic:\n{name_series.str.isalpha()}")
print(f"Is alphanumeric:\n{name_series.str.isalnum()}")
print(f"Is numeric:\n{name_series.str.isnumeric()}")
print(f"Is digit:\n{name_series.str.isdigit()}")

# String slicing with conditions
print(f"\n--- String Slicing with Conditions ---")
# Create a Series with mixed data types
mixed_series = pd.Series(['123abc', '456def', '789ghi', 'abc123', 'def456'])
print(f"Mixed series:\n{mixed_series}")
print(f"Extract first 3 characters:\n{mixed_series.str[:3]}")
print(f"Extract last 3 characters:\n{mixed_series.str[-3:]}")
print(f"Extract middle 3 characters:\n{mixed_series.str[1:4]}")

```

```

# String operations with missing values
print(f"\n--- String Operations with Missing Values ---")
names_with_nan = pd.Series(['John', 'Jane', None, 'Bob', 'Alice'])
print(f"Names with NaN:\n{names_with_nan}")
print(f"Uppercase (handles NaN):\n{names_with_nan.str.upper()}")
print(f"Length (handles NaN):\n{names_with_nan.str.len()}")
print(f"Contains 'a' (handles NaN):\n{names_with_nan.str.contains('a', na

# Advanced string operations
print(f"\n--- Advanced String Operations ---")
# Create a Series with addresses
addresses = pd.Series(['123 Main St, New York, NY 10001', '456 Oak Ave, L
print(f"Addresses:\n{addresses}")
print(f"Extract street numbers:\n{addresses.str.extract(r'^(\d+)')}")
print(f"Extract states:\n{addresses.str.extract(r', ([A-Z]{2}) ')}")
print(f"Extract zip codes:\n{addresses.str.extract(r'(\d{5})$')}")
print(f"Split by comma and get first part:\n{addresses.str.split(',').str

# String operations for data cleaning
print(f"\n--- String Operations for Data Cleaning ---")
dirty_data = pd.Series([' John Smith ', 'jane.doe', 'BOB-WILSON', 'Alic
print(f"Dirty data:\n{dirty_data}")
print(f"Strip whitespace:\n{dirty_data.str.strip()}")
print(f"Replace dots with spaces:\n{dirty_data.str.replace('.', ' ')}")
print(f"Replace hyphens with spaces:\n{dirty_data.str.replace('-', ' ')}")
print(f"Replace @ with space:\n{dirty_data.str.replace('@', ' ')}")
print(f"Replace underscores with spaces:\n{dirty_data.str.replace('_', ' ')}")
print(f"Clean and title case:\n{dirty_data.str.strip().str.replace('[._@-

```

9. Date and Time Operations

```

In [ ]: # Create Series with datetime data
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Create date range
dates = pd.date_range('2024-01-01', periods=30, freq='D')
values = np.random.randint(50, 150, 30)
date_series = pd.Series(values, index=dates, name='Daily Values')

print(f"Date Series:\n{date_series.head(10)}")
print(f"\nIndex type: {type(date_series.index)}")
print(f"Index data type: {date_series.index.dtype}")

# Basic datetime operations
print(f"\n--- Basic Datetime Operations ---")
print(f"Year: {date_series.index.year}")
print(f"Month: {date_series.index.month}")
print(f"Day: {date_series.index.day}")
print(f"Day of week: {date_series.index.dayofweek}")
print(f"Day name: {date_series.index.day_name()}")
print(f"Month name: {date_series.index.month_name()}")

# Date filtering
print(f"\n--- Date Filtering ---")
print(f"January 2024 data:\n{date_series[date_series.index.month == 1]}")

```

```

print(f"First week data:\n{date_series[date_series.index.day <= 7]}")
print(f"Weekend data:\n{date_series[date_series.index.dayofweek >= 5]}")
print(f"Specific date range:\n{date_series['2024-01-10':'2024-01-15']}")

# Date arithmetic
print(f"\n--- Date Arithmetic ---")
print(f"Add 5 days to index:\n{date_series.index + timedelta(days=5)}")
print(f"Subtract 1 week from index:\n{date_series.index - timedelta(weeks=1)}")
print(f"Add 1 month to index:\n{date_series.index + pd.DateOffset(months=1)}")

# Time-based resampling
print(f"\n--- Time-based Resampling ---")
# Create hourly data for resampling example
hourly_dates = pd.date_range('2024-01-01', periods=24, freq='H')
hourly_values = np.random.randint(10, 100, 24)
hourly_series = pd.Series(hourly_values, index=hourly_dates, name='Hourly')

print(f"Hourly data:\n{hourly_series.head()}")
print(f"Daily resample (mean):\n{hourly_series.resample('D').mean()}")
print(f"Daily resample (sum):\n{hourly_series.resample('D').sum()}")
print(f"Daily resample (max):\n{hourly_series.resample('D').max()}")

# Rolling operations on datetime data
print(f"\n--- Rolling Operations on Datetime Data ---")
print(f"3-day rolling mean:\n{date_series.rolling('3D').mean().head(10)}")
print(f"7-day rolling sum:\n{date_series.rolling('7D').sum().head(10)}")
print(f"30-day rolling std:\n{date_series.rolling('30D').std().head(10)}")

# Expanding operations
print(f"\n--- Expanding Operations ---")
print(f"Expanding mean:\n{date_series.expanding().mean().head(10)}")
print(f"Expanding sum:\n{date_series.expanding().sum().head(10)}")

# Date shifting
print(f"\n--- Date Shifting ---")
print(f"Shift forward by 1 day:\n{date_series.shift(1).head(10)}")
print(f"Shift backward by 1 day:\n{date_series.shift(-1).head(10)}")
print(f"Shift by 1 week:\n{date_series.shift(7).head(10)}")

# Period operations
print(f"\n--- Period Operations ---")
# Convert to period
period_series = date_series.to_period('D')
print(f"Converted to period:\n{period_series.head()}")
print(f"Period frequency: {period_series.index.freq}")

# Convert to different frequencies
monthly_period = date_series.to_period('M')
print(f"Monthly period:\n{monthly_period}")

# Time zone operations
print(f"\n--- Time Zone Operations ---")
# Create timezone-aware series
tz_series = date_series.copy()
tz_series.index = tz_series.index.tz_localize('UTC')
print(f"UTC timezone:\n{tz_series.head()}")

# Convert to different timezone
est_series = tz_series.tz_convert('US/Eastern')
print(f"EST timezone:\n{est_series.head()}")

```

```

# Date formatting
print(f"\n--- Date Formatting ---")
print(f"Formatted dates (YYYY-MM-DD):\n{date_series.index.strftime('%Y-%m-%d')}")
print(f"Formatted dates (Month Day, Year):\n{date_series.index.strftime('%B %d, %Y')}")
print(f"Formatted dates (Day/Month/Year):\n{date_series.index.strftime('%d/%m/%Y')}")

# Business day operations
print(f"\n--- Business Day Operations ---")
# Create business day series
bday_series = pd.Series(np.random.randint(100, 200, 20),
                        index=pd.bdate_range('2024-01-01', periods=20),
                        name='Business Values')
print(f"Business day series:\n{bday_series.head(10)}")

# Check if dates are business days
print(f"Is business day:\n{date_series.index.isin(pd.bdate_range('2024-01-01', periods=20))}")

# Business day arithmetic
print(f"Add 5 business days:\n{date_series.index + pd.offsets.BDay(5)}")

# Time delta operations
print(f"\n--- Time Delta Operations ---")
# Create time delta series
time_deltas = pd.Series([timedelta(days=1), timedelta(hours=12), timedelta(
    seconds=45), timedelta(microseconds=500)])
print(f"Time deltas:\n{time_deltas}")

# Operations on time deltas
print(f"Total time:\n{time_deltas.sum()}")
print(f"Mean time:\n{time_deltas.mean()}")
print(f"Convert to seconds:\n{time_deltas.dt.total_seconds()}")

# Date comparison
print(f"\n--- Date Comparison ---")
reference_date = pd.Timestamp('2024-01-15')
print(f"Dates before reference:\n{date_series.index < reference_date}")
print(f"Dates after reference:\n{date_series.index > reference_date}")
print(f"Dates equal to reference:\n{date_series.index == reference_date}")

# Date difference
print(f"Days from reference:\n{(date_series.index - reference_date).days}")

# Seasonal analysis
print(f"\n--- Seasonal Analysis ---")
# Add seasonal data
seasonal_data = np.sin(np.arange(30) * 2 * np.pi / 365) * 10 + 100
seasonal_series = pd.Series(seasonal_data, index=dates, name='Seasonal Data')
print(f"Seasonal data:\n{seasonal_series.head(10)}")

# Group by month
monthly_groups = seasonal_series.groupby(seasonal_series.index.month)
print(f"Monthly means:\n{monthly_groups.mean()}")

# Group by day of week
dow_groups = seasonal_series.groupby(seasonal_series.index.dayofweek)
print(f"Day of week means:\n{dow_groups.mean()}")

# Date range operations
print(f"\n--- Date Range Operations ---")

```

```

# Create date range
date_range = pd.date_range('2024-01-01', '2024-12-31', freq='M')
print(f"Monthly date range:\n{date_range}")

# Check if dates are in range
in_range = date_series.index.isin(date_range)
print(f"Dates in monthly range:\n{in_range}")

# Create custom date range
custom_range = pd.date_range('2024-01-01', periods=10, freq='2W')
print(f"Bi-weekly custom range:\n{custom_range}")

# Working with different date formats
print(f"\n--- Working with Different Date Formats ---")
# String dates
string_dates = ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04', '2024-01-05']
string_series = pd.Series([10, 20, 30, 40, 50], index=string_dates)
print(f"String dates series:\n{string_series}")

# Convert string to datetime
datetime_index = pd.to_datetime(string_series.index)
print(f"Converted to datetime:\n{datetime_index}")

# Parse different date formats
different_formats = ['01/01/2024', '02-Jan-2024', '2024-01-03', 'Jan 04, 2024']
parsed_dates = pd.to_datetime(different_formats)
print(f"Parsed different formats:\n{parsed_dates}")

```

10. Series Aggregation and Statistics

```

In [ ]: # Create comprehensive Series for aggregation and statistics examples
np.random.seed(42)
data = np.random.normal(100, 20, 1000) # Normal distribution
categories = np.random.choice(['A', 'B', 'C', 'D'], 1000)
dates = pd.date_range('2024-01-01', periods=1000, freq='D')

# Create multiple Series for different examples
main_series = pd.Series(data, name='Values')
category_series = pd.Series(categories, name='Category')
date_series = pd.Series(data, index=dates, name='TimeSeries')

# Add some missing values for demonstration
main_series.iloc[100:110] = np.nan
main_series.iloc[500:505] = np.nan

print(f"Main Series (first 10 values):\n{main_series.head(10)}")
print(f"Series length: {len(main_series)}")
print(f"Missing values: {main_series.isnull().sum()}")

# Basic aggregation functions
print(f"\n--- Basic Aggregation Functions ---")
print(f"Count (non-null): {main_series.count()}")
print(f"Sum: {main_series.sum():.2f}")
print(f"Mean: {main_series.mean():.2f}")
print(f"Median: {main_series.median():.2f}")
print(f"Mode: {main_series.mode().values}")
print(f"Standard deviation: {main_series.std():.2f}")
print(f"Variance: {main_series.var():.2f}")

```

```

print(f"Min: {main_series.min():.2f}")
print(f"Max: {main_series.max():.2f}")
print(f"Range: {main_series.max() - main_series.min():.2f}")

# Percentile statistics
print(f"\n--- Percentile Statistics ---")
percentiles = [0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99]
for p in percentiles:
    print(f"{p*100:4.0f}th percentile: {main_series.quantile(p):.2f}")

# Using describe() for comprehensive statistics
print(f"\n--- Comprehensive Statistics (describe) ---")
print(f"Description:\n{main_series.describe()}")

# Custom percentiles
print(f"\n--- Custom Percentiles ---")
custom_percentiles = [0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.8]
print(f"Custom percentiles:\n{main_series.quantile(custom_percentiles)}")

# Distribution statistics
print(f"\n--- Distribution Statistics ---")
print(f"Skewness: {main_series.skew():.4f}")
print(f"Kurtosis: {main_series.kurtosis():.4f}")
print(f"Kurtosis excess: {main_series.kurtosis() - 3:.4f}")

# Robust statistics (less sensitive to outliers)
print(f"\n--- Robust Statistics ---")
print(f"Median absolute deviation: {np.median(np.abs(main_series - main_s
print(f"Interquartile range (IQR): {main_series.quantile(0.75) - main_ser

# Grouped aggregation
print(f"\n--- Grouped Aggregation ---")
# Create a DataFrame for grouped operations
df = pd.DataFrame({
    'values': main_series,
    'category': category_series
})

print(f"Grouped by category - mean:\n{df.groupby('category')['values'].me
print(f"Grouped by category - std:\n{df.groupby('category')['values'].std
print(f"Grouped by category - count:\n{df.groupby('category')['values'].c

# Multiple aggregations at once
print(f"\n--- Multiple Aggregations ---")
agg_results = df.groupby('category')['values'].agg(['count', 'mean', 'std
print(f"Multiple aggregations:\n{agg_results}")

# Custom aggregation functions
print(f"\n--- Custom Aggregation Functions ---")
def range_func(x):
    return x.max() - x.min()

def coefficient_of_variation(x):
    return x.std() / x.mean() if x.mean() != 0 else np.nan

custom_agg = df.groupby('category')['values'].agg([
    'count', 'mean', 'std', range_func, coefficient_of_variation
])
custom_agg.columns = ['Count', 'Mean', 'Std', 'Range', 'CV']
print(f"Custom aggregations:\n{custom_agg}")

```



```

# Time series aggregation
print(f"\n--- Time Series Aggregation ---")
print(f"Daily resampling (mean):\n{date_series.resample('D').mean().head(10)}")
print(f"Weekly resampling (sum):\n{date_series.resample('W').sum().head(10)}")
print(f"Monthly resampling (mean):\n{date_series.resample('M').mean().head(10)}")

# Rolling aggregations
print(f"\n--- Rolling Aggregations ---")
print(f"7-day rolling mean:\n{date_series.rolling(7).mean().head(10)}")
print(f"30-day rolling std:\n{date_series.rolling(30).std().head(10)}")
print(f"7-day rolling min:\n{date_series.rolling(7).min().head(10)}")
print(f"7-day rolling max:\n{date_series.rolling(7).max().head(10)}")

# Expanding aggregations
print(f"\n--- Expanding Aggregations ---")
print(f"Expanding mean:\n{date_series.expanding().mean().head(10)}")
print(f"Expanding sum:\n{date_series.expanding().sum().head(10)}")
print(f"Expanding std:\n{date_series.expanding().std().head(10)}")

# Weighted aggregations
print(f"\n--- Weighted Aggregations ---")
# Create weights
weights = np.random.uniform(0.5, 1.5, len(main_series))
print(f"Weighted mean: {np.average(main_series.dropna(), weights=weights)}")

# Using pandas weighted mean
weighted_series = pd.Series(main_series.values, weights=weights)
print(f"Pandas weighted mean: {weighted_series.mean():.2f}")

# Conditional aggregations
print(f"\n--- Conditional Aggregations ---")
print(f"Mean of values > 100: {main_series[main_series > 100].mean():.2f}")
print(f"Mean of values < 80: {main_series[main_series < 80].mean():.2f}")
print(f"Count of values between 90 and 110: {(main_series >= 90) & (main_series < 110).sum()}")

# Cumulative aggregations
print(f"\n--- Cumulative Aggregations ---")
print(f"Cumulative sum (first 10):\n{main_series.cumsum().head(10)}")
print(f"Cumulative mean (first 10):\n{main_series.expanding().mean().head(10)}")
print(f"Cumulative max (first 10):\n{main_series.cummax().head(10)}")
print(f"Cumulative min (first 10):\n{main_series.cummin().head(10)}")

# Statistical tests
print(f"\n--- Statistical Tests ---")
from scipy import stats

# Normality test
shapiro_stat, shapiro_p = stats.shapiro(main_series.dropna()[:100]) # Li
print(f"Shapiro-Wilk normality test: statistic={shapiro_stat:.4f}, p-value={shapiro_p:.4f}")

# Correlation with time
time_correlation = main_series.corr(pd.Series(range(len(main_series))))
print(f"Correlation with time: {time_correlation:.4f}")

# Outlier detection using IQR
Q1 = main_series.quantile(0.25)
Q3 = main_series.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR

```



```

upper_bound = Q3 + 1.5 * IQR
outliers = main_series[(main_series < lower_bound) | (main_series > upper_bound)]
print(f"Outliers (IQR method): {len(outliers)} values")
print(f"Outlier percentage: {len(outliers) / len(main_series) * 100:.2f}%")

# Z-score outlier detection
z_scores = np.abs((main_series - main_series.mean()) / main_series.std())
z_outliers = main_series[z_scores > 3]
print(f"Z-score outliers (>3 std): {len(z_outliers)} values")

# Summary statistics by groups
print(f"\n--- Summary Statistics by Groups ---")
# Create more complex grouping
df['value_range'] = pd.cut(df['values'], bins=[0, 80, 100, 120, 200],
                           labels=['Low', 'Medium', 'High', 'Very High'])
print(f"Summary by value range:\n{df.groupby('value_range')['values'].describe()}")

# Cross-tabulation
print(f"\n--- Cross-tabulation ---")
crosstab = pd.crosstab(df['category'], df['value_range'])
print(f"Category vs Value Range:\n{crosstab}")

# Pivot table style aggregation
print(f"\n--- Pivot Table Style Aggregation ---")
pivot_agg = df.groupby(['category', 'value_range'])['values'].agg(['count'])
print(f"Pivot aggregation:\n{pivot_agg}")

# Advanced aggregation with custom functions
print(f"\n--- Advanced Aggregation with Custom Functions ---")
def custom_stats(x):
    return pd.Series({
        'count': x.count(),
        'mean': x.mean(),
        'std': x.std(),
        'skew': x.skew(),
        'kurt': x.kurtosis(),
        'q25': x.quantile(0.25),
        'q75': x.quantile(0.75)
    })

advanced_stats = df.groupby('category')['values'].apply(custom_stats)
print(f"Advanced statistics by category:\n{advanced_stats}")

# Memory usage and performance
print(f"\n--- Memory Usage and Performance ---")
print(f"Series memory usage: {main_series.memory_usage()} bytes")
print(f"Series memory usage (deep): {main_series.memory_usage(deep=True)} bytes")
print(f>Data type: {main_series.dtype}")
print(f"Size: {main_series.size}")
print(f"Shape: {main_series.shape}")

```

11. Series Plotting Basics

```

In [ ]: # Import plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

```

```

# Set up plotting style
plt.style.use('default')
sns.set_palette("husl")

# Create various Series for plotting examples
np.random.seed(42)

# Time series data
dates = pd.date_range('2024-01-01', periods=100, freq='D')
time_series = pd.Series(np.cumsum(np.random.randn(100)) + 100, index=dates)

# Categorical data
categories = ['A', 'B', 'C', 'D', 'E']
category_counts = pd.Series([25, 30, 20, 15, 10], index=categories, name='Category Counts')

# Numerical data
numerical_data = pd.Series(np.random.normal(50, 15, 1000), name='Numerical Data')

# Sales data with trends
sales_dates = pd.date_range('2024-01-01', periods=365, freq='D')
sales_data = pd.Series(
    np.sin(np.arange(365) * 2 * np.pi / 365) * 20 + 100 + np.random.randn(365),
    index=sales_dates, name='Daily Sales'
)

print("Series plotting examples with pandas and matplotlib")
print("=" * 50)

# Basic line plot
print("\n--- Basic Line Plot ---")
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Simple line plot
time_series.plot(ax=axes[0, 0], title='Time Series Line Plot', color='blue')
axes[0, 0].set_ylabel('Value')
axes[0, 0].grid(True, alpha=0.3)

# Line plot with custom styling
time_series.plot(ax=axes[0, 1], title='Styled Line Plot',
                 color='red', linestyle='--', marker='o', markersize=4)
axes[0, 1].set_ylabel('Value')
axes[0, 1].grid(True, alpha=0.3)

# Bar plot
category_counts.plot(kind='bar', ax=axes[1, 0], title='Category Bar Plot')
axes[1, 0].set_ylabel('Count')
axes[1, 0].tick_params(axis='x', rotation=45)

# Histogram
numerical_data.plot(kind='hist', ax=axes[1, 1], title='Numerical Data Histogram',
                    bins=30, alpha=0.7, color='orange')
axes[1, 1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

# Different plot types
print("\n--- Different Plot Types ---")
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

```

```

# Line plot
time_series.plot(ax=axes[0, 0], title='Line Plot', color='blue')
axes[0, 0].grid(True, alpha=0.3)

# Bar plot
category_counts.plot(kind='bar', ax=axes[0, 1], title='Bar Plot', color='red')
axes[0, 1].tick_params(axis='x', rotation=45)

# Horizontal bar plot
category_counts.plot(kind='barh', ax=axes[0, 2], title='Horizontal Bar Plot', color='green')

# Histogram
numerical_data.plot(kind='hist', ax=axes[1, 0], title='Histogram', bins=30)
axes[1, 0].set_ylabel('Frequency')

# Box plot
numerical_data.plot(kind='box', ax=axes[1, 1], title='Box Plot', color='red')
axes[1, 1].set_ylabel('Value')

# Density plot (KDE)
numerical_data.plot(kind='density', ax=axes[1, 2], title='Density Plot', color='blue')
axes[1, 2].set_ylabel('Density')

plt.tight_layout()
plt.show()

# Time series specific plots
print("\n--- Time Series Specific Plots ---")
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Time series with trend
sales_data.plot(ax=axes[0, 0], title='Sales Data with Trend', color='blue')
axes[0, 0].set_ylabel('Sales')
axes[0, 0].grid(True, alpha=0.3)

# Rolling mean
rolling_mean = sales_data.rolling(window=30).mean()
sales_data.plot(ax=axes[0, 1], title='Sales with 30-day Rolling Mean', color='blue')
rolling_mean.plot(ax=axes[0, 1], color='red', linewidth=2, label='30-day Rolling Mean')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Seasonal decomposition (simplified)
monthly_avg = sales_data.groupby(sales_data.index.month).mean()
monthly_avg.plot(kind='bar', ax=axes[1, 0], title='Monthly Average Sales', color='green')
axes[1, 0].set_xlabel('Month')
axes[1, 0].set_ylabel('Average Sales')
axes[1, 0].tick_params(axis='x', rotation=0)

# Weekly pattern
weekly_avg = sales_data.groupby(sales_data.index.dayofweek).mean()
weekdays = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
weekly_avg.index = weekdays
weekly_avg.plot(kind='bar', ax=axes[1, 1], title='Weekly Sales Pattern', color='red')
axes[1, 1].set_xlabel('Day of Week')
axes[1, 1].set_ylabel('Average Sales')
axes[1, 1].tick_params(axis='x', rotation=45)

plt.tight_layout()

```

```

plt.show()

# Statistical plots
print("\n--- Statistical Plots ---")
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Histogram with KDE
numerical_data.plot(kind='hist', ax=axes[0, 0], title='Histogram with KDE',
                    bins=30, alpha=0.7, density=True, color='skyblue')
numerical_data.plot(kind='density', ax=axes[0, 0], color='red', linewidth=2)
axes[0, 0].set_ylabel('Density')

# Box plot
numerical_data.plot(kind='box', ax=axes[0, 1], title='Box Plot', color='lightcoral')
axes[0, 1].set_ylabel('Value')

# Q-Q plot (quantile-quantile plot)
from scipy import stats
stats.probplot(numerical_data.dropna(), dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Q-Q Plot (Normal Distribution)')
axes[1, 0].grid(True, alpha=0.3)

# Cumulative distribution
cumulative = numerical_data.sort_values().cumsum() / numerical_data.sum()
cumulative.plot(ax=axes[1, 1], title='Cumulative Distribution', color='purple')
axes[1, 1].set_ylabel('Cumulative Probability')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Custom styling and annotations
print("\n--- Custom Styling and Annotations ---")
fig, ax = plt.subplots(1, 1, figsize=(12, 6))

# Plot with custom styling
time_series.plot(ax=ax, title='Custom Styled Time Series Plot',
                color='darkblue', linewidth=2, marker='o', markersize=4,

# Add annotations
max_val = time_series.max()
max_date = time_series.idxmax()
ax.annotate(f'Peak: {max_val:.2f}',
            xy=(max_date, max_val),
            xytext=(10, 10),
            textcoords='offset points',
            bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow', alpha=0.5),
            arrowprops=dict(arrowstyle='->', connectionstyle='arc3,rad=0'))

# Add trend line
z = np.polyfit(range(len(time_series)), time_series.values, 1)
p = np.poly1d(z)
ax.plot(time_series.index, p(range(len(time_series))), "r--", alpha=0.8,
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_ylabel('Value')

plt.tight_layout()
plt.show()

```

```

# Subplot examples
print("\n--- Subplot Examples ---")
fig, axes = plt.subplots(3, 1, figsize=(12, 15))

# Original data
sales_data.plot(ax=axes[0], title='Original Sales Data', color='blue', al
axes[0].set_ylabel('Sales')
axes[0].grid(True, alpha=0.3)

# Moving averages
sales_data.plot(ax=axes[1], title='Sales with Moving Averages', color='bl
sales_data.rolling(window=7).mean().plot(ax=axes[1], color='red', label='
sales_data.rolling(window=30).mean().plot(ax=axes[1], color='green', labe
axes[1].legend()
axes[1].set_ylabel('Sales')
axes[1].grid(True, alpha=0.3)

# Detrended data
detrended = sales_data - sales_data.rolling(window=30).mean()
detrended.plot(ax=axes[2], title='Detrended Sales Data', color='purple')
axes[2].set_ylabel('Detrended Sales')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Plot customization options
print("\n--- Plot Customization Options ---")
fig, ax = plt.subplots(1, 1, figsize=(12, 6))

# Create a more complex plot
x = pd.date_range('2024-01-01', periods=50, freq='D')
y1 = np.sin(np.arange(50) * 2 * np.pi / 50) * 10 + 50
y2 = np.cos(np.arange(50) * 2 * np.pi / 50) * 10 + 50

series1 = pd.Series(y1, index=x, name='Series 1')
series2 = pd.Series(y2, index=x, name='Series 2')

# Plot multiple series
series1.plot(ax=ax, color='blue', linewidth=2, label='Series 1', marker='
series2.plot(ax=ax, color='red', linewidth=2, label='Series 2', marker='s

# Customize the plot
ax.set_title('Multiple Series Plot with Custom Styling', fontsize=16, fon
ax.set_xlabel('Date', fontsize=12)
ax.set_ylabel('Value', fontsize=12)
ax.legend(fontsize=10, loc='upper right')
ax.grid(True, alpha=0.3, linestyle='--')
ax.set_facecolor('lightgray')

# Add text annotation
ax.text(0.02, 0.98, 'Custom Plot Example', transform=ax.transAxes,
        fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

plt.tight_layout()
plt.show()

# Save plots
print("\n--- Saving Plots ---")

```

```

# Create a simple plot to save
fig, ax = plt.subplots(1, 1, figsize=(10, 6))
time_series.plot(ax=ax, title='Time Series Plot', color='blue', linewidth=2)
ax.grid(True, alpha=0.3)
ax.set_ylabel('Value')

# Save the plot
plt.savefig('time_series_plot.png', dpi=300, bbox_inches='tight')
print("Plot saved as 'time_series_plot.png'")

plt.show()

# Plot parameters and options
print("\n--- Plot Parameters and Options ---")
print("Common plot parameters:")
print("- kind: 'line', 'bar', 'barh', 'hist', 'box', 'density', 'area', 'scatter'")
print("- color: color name or hex code")
print("- title: plot title")
print("- xlabel, ylabel: axis labels")
print("- figsize: tuple (width, height)")
print("- alpha: transparency (0-1)")
print("- linewidth: line thickness")
print("- marker: marker style ('o', 's', '^', etc.)")
print("- markersize: marker size")
print("- grid: True/False for grid lines")
print("- legend: True/False for legend")

# Advanced plotting with seaborn
print("\n--- Advanced Plotting with Seaborn ---")
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Convert Series to DataFrame for seaborn
df_plot = pd.DataFrame({'values': numerical_data, 'category': np.random.choice(categories, len(numerical_data))})

# Histogram with seaborn
sns.histplot(data=df_plot, x='values', ax=axes[0, 0], kde=True)
axes[0, 0].set_title('Seaborn Histogram with KDE')

# Box plot with seaborn
sns.boxplot(data=df_plot, x='category', y='values', ax=axes[0, 1])
axes[0, 1].set_title('Seaborn Box Plot by Category')

# Violin plot
sns.violinplot(data=df_plot, x='category', y='values', ax=axes[1, 0])
axes[1, 0].set_title('Seaborn Violin Plot by Category')

# Time series with seaborn
ts_df = pd.DataFrame({'date': time_series.index, 'value': time_series.values})
sns.lineplot(data=ts_df, x='date', y='value', ax=axes[1, 1])
axes[1, 1].set_title('Seaborn Time Series Plot')
axes[1, 1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

print("\nSeries plotting examples completed!")
print("Key takeaways:")
print("1. Use .plot() method for quick plotting")
print("2. Customize with matplotlib parameters")
print("3. Use seaborn for statistical plots")

```

```
print("4. Save plots with plt.savefig()")
print("5. Combine multiple plots with subplots")
```

Summary

This module covered comprehensive examples for all 11 topics in Pandas Series:

1. **Introduction to Pandas and its ecosystem** - Overview of pandas library and its data structures
2. **Creating Series objects** - Various ways to create Series from different data sources
3. **Series attributes and methods** - Understanding Series properties and basic methods
4. **Indexing and selection** - Accessing data using different indexing methods
5. **Series operations and arithmetic** - Mathematical and logical operations on Series
6. **Handling missing values in Series** - Detection, removal, and filling of missing data
7. **Series methods for data analysis** - Advanced analytical methods and statistical functions
8. **String operations on Series** - Text processing and manipulation using .str accessor
9. **Date and time operations** - Working with datetime data and time series
10. **Series aggregation and statistics** - Comprehensive statistical analysis and grouping
11. **Series plotting basics** - Visualization using matplotlib and seaborn

Key Takeaways:

- Series is a fundamental 1D data structure in pandas
- Powerful indexing and selection capabilities
- Rich set of methods for data manipulation and analysis
- Excellent integration with NumPy and matplotlib
- Essential for data cleaning, analysis, and visualization

Next Steps:

- Practice with real-world datasets
- Explore DataFrame operations (Module 5)
- Learn advanced data manipulation techniques
- Apply Series operations in data science projects